

Hash Functions

A hash function, $h : X \rightarrow Y$, assigns a hash value $h(x)$ to some data x (x can be any type of data including messages, passwords, strings, etc...). The hash values have a fixed size, but the data can have any size.

Let X be the set of all possible input data (messages).

Let Y be the set of all possible hash values (message digests).

A pair $(x, y) \in X \times Y$ is a valid pair under a hash function h if $h(x) = y$.

Example: The hash function $h(x) = x \bmod n$, where $n = 128$, from the set of all possible binary input strings to the set of all 7-bit message digests.

The pair $(111100001100, 0001100)$ is valid because $3852 \bmod 128 = 12$.

The pair $(010101010101, 1111111)$ is not valid because $1365 \bmod 128 = 85 \neq 127$.

For a hash function, $h : X \rightarrow Y$ to be considered to be secure it must be difficult to solve the following three problems:

1. **Preimage:** Given $y \in Y$, find $x \in X$ such that $h(x) = y$.
2. **Second Preimage:** Given $x \in X$, find $x' \in X$ such that $h(x') = h(x)$ and $x' \neq x$.
3. **Collision:** Find $x, x' \in X$ such that $h(x') = h(x)$ and $x' \neq x$.

Example: The hash function $h(x) = x \bmod n$, where $n = 128$, from the set of all possible binary input strings to the set of all 7-bit message digests is not secure.

The ideal case for a hash function is a “Random Oracle” where the only way to compute $h(x)$ is to query the oracle.

Let $F^{X,Y}$ denote the set of functions from $X \rightarrow Y$.

Let $|X| = N$ and $|Y| = M$. Then $|F^{X,Y}| = M^N$

If a hash function “behaves” like a random oracle then the probability of solving the preimage problem is $P(h(x) = y) = \frac{1}{M}$.

Secure Hash Algorithm (SHA)

Secure hash algorithms use sets of cryptographic hash functions; one-way functions that map input data of variable size to bit strings of fixed size. These functions are not invertible so the only way to attempt to obtain the input data from the output bit string is to brute force search all possible input data. Cryptographic hash functions also have a very low probability of collision.

The NSA’s “SHA-256” is a popular algorithm currently in use. The input data is broken up into 512-bit chunks and put through a “modular 2^{32} mathematical meat grinder” involving the fractional part of the square and cube roots of primes, mixing bits, and compressions. At the end, a 256-bit string is output (which is usually displayed in hex using 64 hexits).

Examples:

SHA-256 (“INFO39921”) =
3eb6f83ce90f0047b93ef747b65c7c6127b94d6f2e53d442598836ab9e6ddd4b

SHA-256(“All your base are belong to us”) =
c46ec1b18ce8a878725a37e780dfb7351f68ed2e194c79fbc6aebee1a667975d

SHA-256(“”) =
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

The only known way of obtaining the input data string (x) for any of these SHA-256 hash values (y) is to run the algorithm. In this way the SHA-256 “behaves” like a random oracle. Therefore, the probability of

solving the preimage problem is $P(h(x) = y) = \frac{1}{2^{256}}$. This is often stated as a preimage security of

$t=256$, because it would take about $2^t = 2^{256}$ steps to brute force a search for a preimage.

Exercises:

1. Compute the hash value (in binary) for $x=10110111$ using the following hash functions:
 - a) $h(x) = x \bmod 32$
 - b) $h(x) = x \bmod 31$
 - c) $h(x) = x \bmod 9$
2. Determine the SHA-256 hash values (in hexadecimal) of the following inputs:
 - a) "299792458 mps"
 - b) "TK-421, do you copy?"
3. What is the preimage security of:
 - a) $h(x) = f(x) \bmod 3$
 - b) $h(x) = f(x) \bmod 64$
 - c) $h(x) = f(x) \bmod 97$
 - d) SHA-512(x)
4. Which pairs $(x, y) \in X \times Y$ are valid for $h(x) = x \bmod 7$:
 - a) (1000100, 101)
 - b) (11001, 101)
 - c) (101010101010, 100)
5. How many different hash functions are there from:
 - a) 16-bit data to 8-bit strings?
 - b) 4-digit PIN numbers to 10-bit strings?
 - c) 3-character lowercase passwords to 7-bit strings?

Answers:

1.

- a) 10111
- b) 11100
- c) 00011

2.

- a) 65f1420564305ba1e0a78d3cc845a295905320480068fec76b4a9816205644c9
- b) af6c866c00d65a5c5668df12f779588f5a4d0f25af1741be1635a5b7943ad223

3.

- a) $t=1.6$
- b) $t=6.0$
- c) $t=6.6$
- d) $t=512$

4.

- a) valid
- b) not valid
- c) not valid

5.

- a) 256^{65536}
- b) 1024^{10000}
- c) 128^{17576}